6주차

(1) LoRA

1. Introduction



논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- **Parameter-Efficient Fine-Tuning (PEFT)**: 대규모 사전학습 언어모델을 적은 수의 파라미터만 학습하여 downstream task에 효율적으로 adaptation하는 방법
- 기존 Full Fine-Tuning은 모든 파라미터를 업데이트하기 때문에 모델이 커질수록 학습·저장·배포 비용이 크게 증가
- 특히 GPT-3 175B처럼 거대한 모델은 task마다 별도의 fine-tuned model을 저장하고 운영하기 어려움

해당 task에서 기존 연구 한계점

Full Fine-Tuning

- 모든 pretrained parameter를 업데이트하므로 task마다 원본 모델과 동일한 규모의 파라미터를 저장해야 함
- 대규모 모델에서는 GPU memory와 storage 비용이 매우 큼

Adapter

- pretrained model을 freeze하고 작은 adapter layer만 추가하여 학습 파라미터를 줄임
- 하지만 기존 layer 사이에 새로운 연산이 추가되어 **inference latency가 증가**

Prefix/Prompt Tuning

- 입력이나 hidden activation에 학습 가능한 special token을 추가

- optimization이 어렵고, special token이 입력 공간을 차지하여 **사용 가능한 sequence length가 감소**

기존 PEFT 방법은 효율성을 높이는 대신 성능이나 inference efficiency를 일부 희생하는 문제가 존재

논문의 contributions

- **LoRA (Low-Rank Adaptation)** 제안
  - pretrained weight는 freeze하고, downstream adaptation에서 발생하는 weight update `ΔW`만 low-rank matrix로 학습
- `ΔW = BA`로 분해하여 거대한 weight matrix 전체가 아닌 작은 `A`, `B`만 업데이트
- GPT-3 175B 기준
  - trainable parameter 최대 **10,000배 감소**
  - GPU memory requirement 약 **3배 감소**
- RoBERTa, DeBERTa, GPT-2, GPT-3에서 Full Fine-Tuning과 동등하거나 더 높은 성능 달성
- 학습된 LoRA weight를 기존 weight에 합칠 수 있어 **추가 inference latency가 없음**
- 실험을 통해 downstream adaptation의 weight update가 실제로 매우 낮은 **intrinsic rank**를 가질 수 있음을 분석 2106.09685v2

2. Related Work



Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

Transformer Language Model

- BERT, GPT-2, GPT-3 등 대규모 Transformer를 먼저 pretraining하고 downstream task에 fine-tuning하는 방식이 NLP의 주요 paradigm으로 자리잡음
- 모델 규모가 커질수록 성능은 향상되지만 fine-tuning 비용 역시 크게 증가

Fine-Tuning / Prompt Engineering

- Prompting은 모델 parameter를 수정하지 않아 효율적이지만 성능이 prompt 구성에 크게 의존

- Fine-Tuning은 높은 성능을 얻을 수 있지만 대규모 모델에서는 학습 및 checkpoint 저장 비용이 큼

Adapter

- pretrained model 사이에 작은 trainable module을 추가
- 학습 parameter는 줄일 수 있지만 추가 layer 연산으로 inference latency 발생

Prefix / Prompt Tuning

- 모델 대신 input embedding이나 hidden activation을 학습
- 적은 parameter로 adaptation할 수 있지만 special token이 context를 차지하고 optimization이 어려울 수 있음

LoRA의 차별점

- Adapter처럼 parameter-efficient하면서도 학습된 matrix를 기존 weight에 **merge 가능**
- 따라서 **추가 inference latency가 없고 sequence length도 감소하지 않음**
- 기존 low-rank 연구와 달리 모델 자체를 low-rank로 만드는 것이 아니라 **frozen pretrained model의 update `ΔW`에 low-rank constraint를 적용**

3. 제안 방법론



Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

- 핵심 가정: **downstream task에 adaptation할 때 필요한 weight 변화 `ΔW`는 낮은 intrinsic rank를 가진다.**
- 기존 Full Fine-Tuning:
  - `W = W₀ + ΔW`
  - `ΔW` 전체를 자유롭게 학습
- LoRA:

    - $W = W_0 + BA$
    - pretrained weight W_0는 **freeze**
    - 작은 low-rank matrix A, B만 **train**
    - $r << min(d, k)$로 설정하여 trainable parameter를 크게 감소
- Forward pass:
    - $h = W_0x + BAx$
- 초기화:
    - A: Random Gaussian
    - B: Zero
    - 따라서 학습 시작 시 $BA = 0$이므로 pretrained model과 동일한 상태에서 시작
- **Transformer 적용**
    - Self-Attention의 W_q, W_k, W_v, W_o 등에 적용 가능
    - 논문의 주요 실험에서는 attention weight만 adaptation하고 MLP는 freeze
    - 대부분의 실험에서 W_q, W_v**에 LoRA 적용** 2106.09685v2
- **Inference**
    - 학습 완료 후 $W = W_0 + BA$로 미리 merge
    - inference 시 기존 모델과 동일하게 계산하므로 **추가 latency가 없음**
    - task 변경 시 작은 LoRA parameter만 교체하면 됨

Contribution

- **Parameter Efficiency**
    - GPT-3 175B에서 checkpoint 크기 약 **350GB → 35MB**
- **Memory Efficiency**
    - Training VRAM 약 **1.2TB → 350GB**
- **Training Efficiency**
    - GPT-3 175B에서 Full Fine-Tuning 대비 training throughput 약 **25% 향상**
- **No Inference Overhead**
    - BA를 기존 weight에 merge할 수 있어 추가 연산 불필요

- **Efficient Task Switching**
  - 하나의 pretrained model을 공유하고 task별 작은 LoRA parameter만 저장하여 빠르게 task 전환 가능

4. 실험 및 결과



Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- **RoBERTa base / large, DeBERTa XXL**
  - GLUE benchmark
  - MNLI, SST-2, MRPC, CoLA, QNLI, QQP, RTE, STS-B
- **GPT-2 medium / large**
  - E2E NLG Challenge
  - DART
  - WebNLG
- **GPT-3 175B**
  - WikiSQL: NL → SQL
  - MNLI: 자연어 추론
  - SAMSum: 대화 요약
- NLU와 NLG를 모두 포함하여 다양한 모델 크기와 task에서 LoRA를 검증

Baseline

- Full Fine-Tuning
- Fine-Tuning Top 2 Layers
- BitFit
- Prefix-Embedding Tuning
- Prefix-Layer Tuning
- Adapter 계열

- LoRA

결과

RoBERTa / DeBERTa

- RoBERTa base: Full FT `86.4` → LoRA `87.2`
- RoBERTa large: Full FT `88.9` → LoRA `89.0`
- DeBERTa XXL: Full FT `91.1` → LoRA `91.3`
- 매우 적은 trainable parameter로 Full Fine-Tuning과 동등하거나 더 높은 성능 달성

GPT-2

- GPT-2 Medium E2E 기준
  - Full FT: **354.92M parameters / BLEU 68.2**
  - LoRA: **0.35M parameters / BLEU 70.4**
- 약 1/1000 수준의 trainable parameter로 더 높은 BLEU 달성

GPT-3 175B

- Full FT: WikiSQL `73.8`, MNLI `89.5`
- LoRA 4.7M: WikiSQL `73.4`, MNLI `91.7`
- LoRA 37.7M: WikiSQL `74.0`, MNLI `91.6`
- Full Fine-Tuning의 175B parameter 전체를 학습하지 않고도 동등하거나 더 높은 성능 달성

Rank Ablation

- `Wq`, `Wv`에 LoRA를 적용하면 `r=1~4`처럼 매우 작은 rank에서도 높은 성능
- WikiSQL (`Wq`, `Wv`)
  - `r=1`: 73.4
  - `r=4`: 73.7
  - `r=8`: 73.8
  - `r=64`: 73.5

- rank를 크게 늘려도 성능 향상이 거의 없음 → adaptation에 필요한 ΔW가 실제로 **매우 낮은 intrinsic rank**를 가진다는 가설을 뒷받침

Weight 분석

- ΔW는 pretrained weight와 완전히 새로운 feature를 만드는 것이 아니라, **pretraining에서 이미 학습되었지만 강조되지 않았던 방향을 downstream task에 맞게 증폭**하는 것으로 관찰됨

5. 결론 (배운점)



연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

연구의 의의

- 거대한 pretrained model 전체를 fine-tuning하지 않아도 **task adaptation에 필요한 변화만 low-rank update로 학습**하면 충분할 수 있음을 보임
- Full Fine-Tuning 수준의 성능을 유지하면서 trainable parameter, memory, storage 비용을 크게 절감
- Adapter와 달리 inference latency를 추가하지 않는다는 실용적 장점도 존재

좋았던 점

- 기존 weight 자체가 아닌 **fine-tuning 과정에서 발생하는 ΔW의 구조**에 주목했다는 점
- RoBERTa부터 GPT-3 175B까지 다양한 규모의 모델과 NLU/NLG task에서 검증
- 성능 비교에 그치지 않고 rank 및 singular subspace 분석을 통해 **왜 작은 rank로도 충분한지** 실험적으로 분석
- **한계점 / 아쉬운 점**
  - 어떤 weight matrix에 LoRA를 적용할지 여전히 heuristic하게 결정하며, 논문에서도 보다 principled한 선택 방법을 future work로 제시
  - 모든 task에서 작은 rank가 충분하다고 보장할 수 없음. 특히 pretraining과 크게 다른 task에서는 더 높은 rank나 Full Fine-Tuning이 필요할 가능성이 있음
  - LoRA weight를 base weight에 merge하는 방식에서는 서로 다른 task의 LoRA를 사용하는 입력들을 하나의 batch로 처리하기 까다로움
- **배운점**
  - Fine-Tuning은 반드시 **모든 parameter를 업데이트해야 하는 과정이 아님**

- 대규모 pretrained model이 이미 풍부한 feature를 학습했다면 downstream adaptation은 전체 모델을 다시 학습하기보다 **필요한 feature direction을 작은 update로 조정하는 문제**로 볼 수 있음
- LoRA의 핵심은 단순한 parameter compression이 아니라 ΔW**의 low intrinsic rank를 이용한 효율적인 adaptation**에 있음

(2) YOLO

1. Introduction



논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

Object Detection

- 이미지 속에 어떤 객체가 있는지(classification)와 어디에 있는지(localization)를 동시에 예측하는 Computer Vision task
- 자율주행, 보조기기, 로봇 등에서는 정확도뿐 아니라 **실시간으로 객체를 탐지할 수 있는 속도**가 중요함
- 이 논문은 기존의 복잡한 detection pipeline을 하나의 neural network로 통합한 YOLO (You Only Look Once)를 제안
- Object Detection을 여러 단계의 처리 과정이 아니라 **이미지 전체에서 bounding box와 class probability를 직접 예측하는 하나의 regression problem**으로 정의함

해당 task에서 기존 연구 한계점

기존 Object Detection은 기본적으로 **classifier를 detection에 활용하는 방식**

- **DPM**: 이미지의 여러 위치와 scale을 sliding window 방식으로 탐색하면서 classifier 실행
- **R-CNN**: 먼저 region proposal로 객체 후보 영역을 생성한 뒤 각 영역을 classifier로 분류하고, bounding box 보정 및 중복 제거 등의 후처리를 수행

이처럼 **region proposal → feature extraction → classification → bounding box refinement → post-processing** 등 여러 단계가 분리되어 있음

- 각 component를 개별적으로 학습하고 조정해야 하므로
  - pipeline이 복잡함
  - 전체 시스템을 end-to-end로 최적화하기 어려움
  - 하나의 이미지에 여러 번 모델을 적용해야 해 inference가 느림
- 또한 기존 region-based 방법은 이미지의 일부 영역을 중심으로 판단하기 때문에 **전체 이미지의 contextual information을 충분히 활용하지 못할 수 있음**
  - 논문에서는 Fast R-CNN이 background를 object로 잘못 탐지하는 문제가 있으며, YOLO는 전체 이미지를 보기 때문에 background error가 더 적다고 설명

논문의 contributions

- **YOLO: Unified Object Detection 제안**
  - Object Detection을 하나의 **regression problem**으로 재정의
  - 하나의 CNN이 전체 이미지를 **한 번만 보고**, 여러 bounding box와 각 box의 class probability를 동시에 예측
  - detection의 여러 component를 하나의 network로 통합하여 **end-to-end training** 가능
- **Real-Time Object Detection**
  - 복잡한 region proposal 및 반복적인 classification 과정이 필요하지 않아 매우 빠른 inference 가능
  - YOLO: **45 FPS**
  - Fast YOLO: **155 FPS**
  - 실시간 detection이 가능하면서 기존 real-time detector보다 높은 정확도를 달성
- **Global Context 활용**
  - 이미지 일부가 아니라 **전체 이미지를 한 번에 입력**받아 객체의 appearance뿐 아니라 주변 context까지 함께 학습
  - 기존 Fast R-CNN보다 background false positive가 적음
- **높은 Generalization**

    - 자연 이미지로 학습한 후 artwork처럼 다른 domain에 적용했을 때 DPM, R-CNN보다 높은 generalization 성능을 보임
    - 이미지 전체에서 객체의 크기·형태·주변 관계를 함께 학습하기 때문이라고 설명
- **Trade-off**
    - 속도와 generalization에서는 강점을 보이지만 당시 state-of-the-art detector보다 전체 정확도는 낮음
    - 특히 **작은 객체를 정확하게 localization하는 데 어려움**이 있으며, localization error가 YOLO의 주요 한계로 제시됨

기존 Object Detection이 여러 단계의 복잡한 pipeline으로 객체를 찾았다면, YOLO는 **이미지 전체 → bounding box + class probability를 하나의 network가 직접 예측하도록 통합**하여 Object Detection을 빠른 end-to-end regression 문제로 바꾼 연구이다.